差分法与迭代法求解泊松方程

赵智宇　　孙全超

电子邮件: 12110814@mail.sustech.edu.cn　　电子邮件: 12110811@mail.sustech.edu.cn

2024年6月5日

1 背景介绍

泊松方程在各种科学和工程领域中都有广泛的应用， 可以用于描述和解决许多关于流场演化的实际问题， 如电势分布、热传导等问题。通过求解泊松方程，我们可以对相应的自然现象有深入的理解， 对工程建设有更准确的演算。 而由于泊松方程在某些情况下求解困难或者低成本的解析求解， 关于泊松方程以及泊松方程相关变式快速、 高效的数值求解也一直是广受关注的问题。 本项目通过差分法和迭代法（包括松弛迭代）等方法求解泊松方程，并在流体的模拟中进行应用。

泊松方程的一般形式为

$$\nabla^2\phi(\boldsymbol{x}) = g(\boldsymbol{x}) \tag{1}$$

其中，∇为拉普拉斯算子，表示标量场的二阶空间导数，$\phi(\boldsymbol{x})$是待求解的标量场，$g(\boldsymbol{x})$是标量场的源项或驱动项。本项目主要聚焦在如何通过数值的方法求解二维泊松方程。二维泊松方程的形式如下：

$$\frac{\partial^2 u(x,y)}{\partial x^2} + \frac{\partial^2 u(x,y)}{\partial y^2} - g(x,y) = 0 \tag{2}$$

2 差分法

利用二阶导数的中心差分法，得到二维泊松方程的差分形式：

$$\frac{u(x+\Delta x,y)+u(x-\Delta x,y)-2u(x,y)}{(\Delta x)^2} + \frac{u(x,y+\Delta y)+u(x,y-\Delta y)-2u(x,y)}{(\Delta y)^2} - g(x,y) = 0$$

取$\Delta x = \Delta y = h$，将x和y坐标离散化到$N_i \times N_j$个点。记 $x_i = ih, y_j = jh(i,j=0,1,\ldots,N_{i,j}-1)$，$u_{i,j} = u(x_i,y_j), g_{i,j} = g(x_i,y_j)$，得到显式公式：

$$u_{i,j} = \frac{1}{4}[u_{i-1,j} + u_{i+1,j} + u_{i,j-1} + u_{i,j+1} - h^2 g_{i,j}] \tag{3}$$

我们课上学过热传导方程可以用FTCS法， 因为其时间是一阶导的， 可以在时间坐标上做误差精度为1的向前差分。但二维泊松方程不同，它对两个维度都是二阶导的，因此在两维度上都做误差精度为2的中心差分。 这意味着它不能由初始条件出发遍历坐标网络轻松得到结果， 而是在每个坐标点上都形成一个方程组，从而必须求解一个(N_iN_j)维的矩阵。下面我们按第一类边界条件分析。

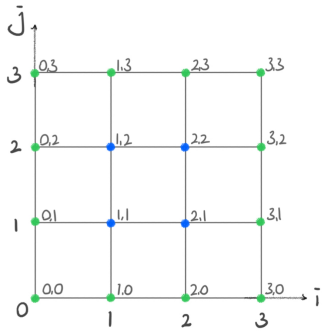


图 1. 求解区域的离散网格示例，蓝色点待求解的未知点，绿色点为边界上的已知点。

以图1为例，记已知点$u_{i,j}=\varphi_{i,j}$，其差分方程为：

$$\begin{pmatrix} -4 & 1 & 1 & 0 \\ 1 & -4 & 0 & 1 \\ 1 & 0 & -4 & 1 \\ 0 & 1 & 1 & -4 \end{pmatrix} \begin{pmatrix} u_{1,1} \\ u_{2,1} \\ u_{1,2} \\ u_{2,2} \end{pmatrix} = \begin{pmatrix} h^2 g_{1,1} - \varphi_{1,0} - \varphi_{0,1} \\ h^2 g_{2,1} - \varphi_{2,0} - \varphi_{3,1} \\ h^2 g_{1,2} - \varphi_{1,3} - \varphi_{0,2} \\ h^2 g_{2,2} - \varphi_{2,3} - \varphi_{3,2} \end{pmatrix}$$

同理外推，在$(N_i+2)\times(N_j+2)$的网格下，差分方程为：

$$\begin{pmatrix} -4 & 1 & \cdots & 1 & & & & & & \\ 1 & \ddots & \ddots & & \ddots & & & & & \\ \vdots & \ddots & -4 & 0 & & 1 & & & & \\ 1 & & 0 & -4 & 1 & & \ddots & & & \\ & \ddots & & 1 & \ddots & \ddots & & \ddots & & \\ & & 1 & & \ddots & -4 & 0 & & 1 & \\ & & & \ddots & & 0 & -4 & 1 & \vdots & \\ & & & & \ddots & & 1 & \ddots & \ddots & \\ & & & & & 1 & \cdots & \ddots & -4 & \end{pmatrix} \begin{pmatrix} u_{1,1} \\ \vdots \\ u_{N_i,1} \\ u_{1,2} \\ \vdots \\ u_{N_i,2} \\ u_{1,3} \\ \vdots \\ u_{N_i,N_j} \end{pmatrix} = h^2 \begin{pmatrix} g_{1,1} \\ \vdots \\ g_{N_i,1} \\ g_{1,2} \\ \vdots \\ g_{N_i,2} \\ g_{1,3} \\ \vdots \\ g_{N_i,N_j} \end{pmatrix} - \begin{pmatrix} \varphi_{1,0} \\ \vdots \\ \varphi_{N_i,0} \\ 0 \\ \vdots \\ 0 \\ \varphi_{1,N_j+1} \\ \vdots \\ \varphi_{N_i,N_j+1} \end{pmatrix} - \begin{pmatrix} \varphi_{0,1} \\ \vdots(0) \\ \varphi_{N_i+1,1} \\ \varphi_{0,2} \\ \vdots(0) \\ \varphi_{N_i+1,2} \\ \varphi_{0,3} \\ \vdots \\ \varphi_{N_i+1,N_j} \end{pmatrix}$$

解该方程组所需的算力会随着N_iN_j的增大而急剧增大，因此我们尝试了利用*Smart Jacobi*等迭代法来加快矩阵计算的速度。

方法代码详见附件*DifferencePoisson1.m* 和 *DiffSmartJPoisson1.m*.

3 迭代法

相较于求解如此高维的矩阵，更好的办法是使用迭代法。经验证明（课上学过），随着数据点的增多，迭代法的时间成本的是相对增长较缓的，即更加高效。

3.1 高斯-赛德尔（Gauss-Seidel）迭代法

我们学过使用*Gauss-Seidel*迭代法求解矩阵的逆（即$Ax=b$的解），主要思想是在一轮迭代中随时将新值投入到下一个新值的计算中。这个思想也可以被应用到偏微分方程的数值求解中，如图2所示。

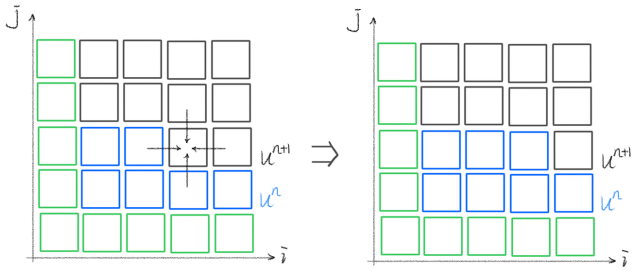


图 2. *Gauss-Seidel*法的原理示意图。绿色为已知的边界点，蓝色为第$n+1$轮迭代出的值，黑色为第n轮的值。

迭代所用的的数值格式由显式公式而来，变成隐式迭代的形式：

$$u_{i,j}^{(n+1)} = \frac{1}{4}[u_{i-1,j}^{(n+1)} + u_{i+1,j}^{(n)} + u_{i,j-1}^{(n+1)} + u_{i,j+1}^{(n)} - h^2 g_{i,j}] \tag{4}$$

当然，我们还需要给待求解的每个点都赋予一个初始（猜测）值，用于启动迭代。边界点上的函数值由第一类边界条件给出，但内部点的值怎么办呢？我们可以将已知的所有边界点函数值u_{bp}取平均，来作为内部所有待求解点的共同初值u_{ip}^0.

$$u_{ip}^0 = \frac{1}{N_{bp}} \sum u_{bp} \tag{5}$$

方法代码详见附件$GaussSeidelPoisson1.m$.

3.2 松弛迭代法

有什么办法能进一步加快迭代的收敛速度呢？ 我们选择使用松弛法[1]， 这是对$Gauss\text{-}Seidel$法的一种改良。具体想法是不再直接将求出的值赋给点，而是将求出的值和这个点原来的值以一定比例“调和”（加权相加），调和参数即松弛变量$\omega\,(\omega\in(0,2))$。基于上述思想，重写隐式差分公式：

$$u_{i,j}^{(n+1)}=(1-\omega)\,u_{i,j}^{(n)}+\frac{1}{4}\omega\big[u_{i-1,j}^{(n+1)}+u_{i+1,j}^{(n)}+u_{i,j-1}^{(n+1)}+u_{i,j+1}^{(n)}-h^2g_{i,j}\big] \tag{6}$$

一般选取$\omega>1$，即超松弛迭代。随着网格精度增大，最佳松弛变量ω也越来越接近2.

方法代码详见附件$RelaxationPoisson1.m$.

3.3 松弛法在第二类边界条件下的实现

在第二类边界条件下， 我们不知道边界点的函数值， 只知道边界上函数值（垂直于边界）的偏导数$\frac{\partial u_{i,j}}{\partial x}=\psi_{i,j}$ or $\frac{\partial u_{i,j}}{\partial y}=\psi_{i,j}$. 在四个边界处，利用向前差分法可以得到关系：

$$\begin{aligned}u_{i,1}&=u_{i,0}+h\psi_{i,0}\\u_{1,j}&=u_{0,j}+h\psi_{0,j}\end{aligned}$$

$$\begin{aligned}u_{i,N_j-2}&=u_{i,N_j-1}-h\psi_{i,N_j-1}\\u_{N_i-2,j}&=u_{N_i-1,j}-h\psi_{N_i-1,j}\end{aligned}$$

结合上式，通过代换$u_{i,0}$，$y=0$边界上的差分公式改写为

$$u_{i,1}^{(n+1)}=(1-\omega)\,u_{i,1}^{(n)}+\frac{\omega}{4}\big[u_{i-1,1}^{(n+1)}+u_{i+1,1}^{(n)}+\big(u_{i,1}^{(n+1)}-h\psi_{i,0}\big)+u_{i,2}^{(n)}-h^2g_{i,1}\big]$$

即

$$u_{i,1}^{(n+1)}=\frac{4}{4-\omega}\left\{(1-\omega)\,u_{i,1}^{(n)}+\frac{\omega}{4}\big[u_{i-1,1}^{(n+1)}+u_{i+1,1}^{(n)}+u_{i,2}^{(n)}-h\psi_{i,0}-h^2g_{i,1}\big]\right\}$$

其余三边同理可得：

$$u_{1,j}^{(n+1)}=\frac{4}{4-\omega}\left\{(1-\omega)\,u_{1,j}^{(n)}+\frac{\omega}{4}\big[u_{1,j-1}^{(n+1)}+u_{1,j+1}^{(n)}+u_{2,j}^{(n)}-h\psi_{0,j}-h^2g_{1,j}\big]\right\}$$

$$u_{i,N_j-2}^{(n+1)}=\frac{4}{4-\omega}\left\{(1-\omega)\,u_{i,N_j-2}^{(n)}+\frac{\omega}{4}\big[u_{i-1,N_j-2}^{(n+1)}+u_{i+1,N_j-2}^{(n)}+u_{i,N_j-3}^{(n+1)}+h\psi_{i,N_j-1}-h^2g_{i,N_j-2}\big]\right\}$$

$$u_{N_i-2,j}^{(n+1)}=\frac{4}{4-\omega}\left\{(1-\omega)\,u_{N_i-2,j}^{(n)}+\frac{\omega}{4}\big[u_{N_i-2,j-1}^{(n+1)}+u_{N_i-2,j+1}^{(n)}+u_{N_i-3,j}^{(n+1)}+h\psi_{N_i-1,j}-h^2g_{N_i-2,j}\big]\right\}$$

将边界上的差分公式改成上述式， 内部点仍同3.1节的差分公式进行实现。 这里我们将所有点的迭代初值设定为$u_{ip}^0=0$.

方法代码详见附件$RelaxationPoisson2.m$.

4 实例与对比结果

4.1 四种方法的测试对比

在本节中，我们通过一些实例横向对比上述四种数值求解泊松方程方法的计算速度和误差。

其中，“普通差分法”所用的是Matlab内置的“左除”来进行矩阵求逆。我们以“普通差分法”作为误差基准，即计算误差时考虑的是其他方法同“普通差分法”的相对误差（求解域内每个点的相对误差绝对值的平均值）。

4.1.1 无内部热源的稳定温度场

一个无内部热源的二维稳定温度场$T(x,y)$,$x,y\in[0,10]$满足如下方程：

$$\frac{\partial^2 T(x,y)}{\partial x^2}+\frac{\partial^2 T(x,y)}{\partial y^2}=0 \tag{7}$$

源项函数$g(x,y)=0$.

边界条件：$T(x,0)=T(x,10)=T(0,y)=0$, $T(y,10)=100$.

方法参数：$h=0.1$, 松弛变量$\omega=1.8$.

测试结果如下：

表格 1. 无内部热源的稳定温度场模型下的四方法对比

	普通差分法	差分法+SmartJ法	GaussSeidel迭代法	松弛迭代法
计算用时	2.1647s	64.0306s	0.14525s	0.11441s
相对误差	-	10.01215	1.06339	0.04197

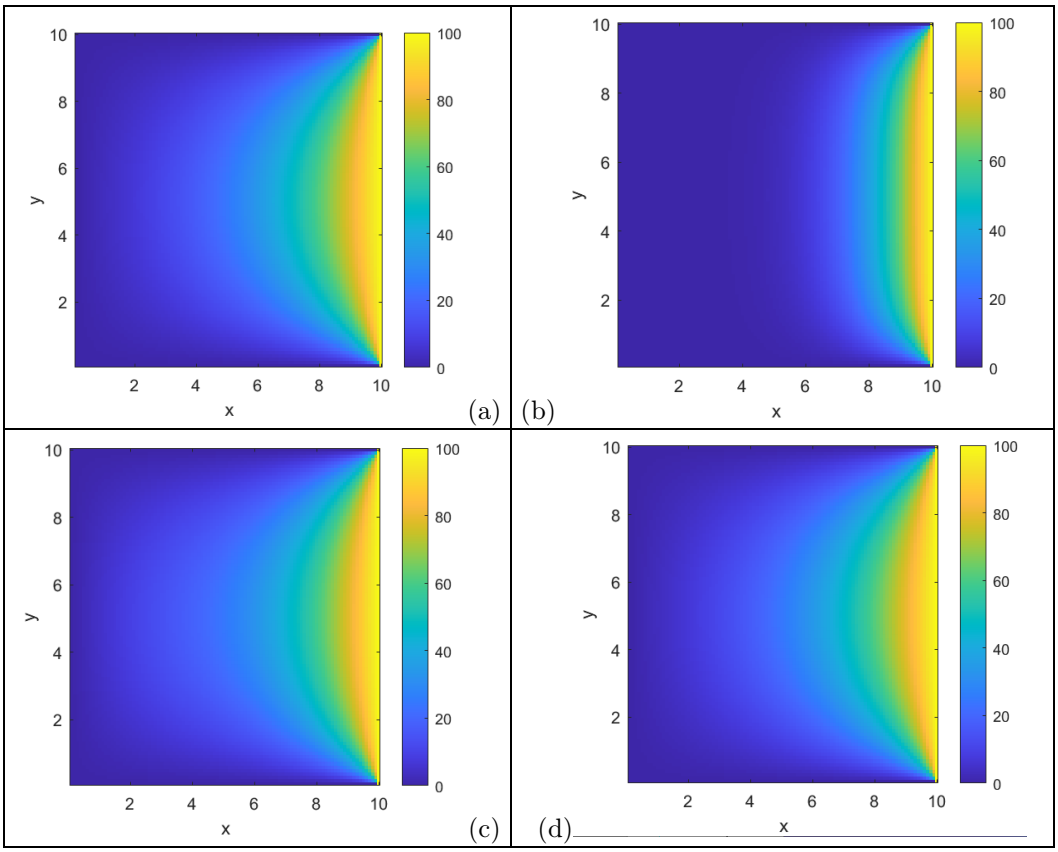


图 3. 无内部热源的稳定温度场模型-四方法的计算值，(a)普通差分法，(b)差分+SmartJ，(c)GaussSeidel迭代，(d)松弛迭代法.

出乎意料地，使用了Smart-Jacobi法求解矩阵的差分法居然大幅增加了计算耗时，且准确性较差。但经分析，我们发现了问题所在：需要求逆的矩阵在绝大部分行都是对角不占优的（对角元：-4，其余元相加：4），因此矩阵的谱半径很接近1，导致收敛的速度急剧减慢了。矩阵维度越大时，这个效应也越明显。

测试代码详见附件*Test1.m*.

4.1.2 给定内部热源功率空间分布的稳定温度场

一个内源热功率的空间分布函数为$Q(x,y)=0.8\,e^{-\left(\frac{(x-5)^2}{0.2}+\frac{(y-5)^2}{0.6}\right)}$的二维稳定温度场$T(x,y)$,$x,y\in[0,10]$满足如下方程：

$$\frac{\partial^2 T(x,y)}{\partial x^2}+\frac{\partial^2 T(x,y)}{\partial y^2}+0.8\,e^{-\left(\frac{(x-5)^2}{0.2}+\frac{(y-5)^2}{0.6}\right)}=0 \tag{8}$$

源项函数：$g(x,y)=-0.8\,e^{-\left(\frac{(x-5)^2}{0.2}+\frac{(y-5)^2}{0.6}\right)}$.

边界条件：$T(x,0)=T(x,10)=T(0,y)=T(y,10)=0$.

方法参数：$h=0.1$, 迭代的容许误差$e=10^{-3}$, 松弛变量$\omega=1.8$.

表格 2. 给定内部热源功率空间分布的稳定温度场模型下的四方法对比

	普通差分法	差分法+SmartJ法	GaussSeidel迭代法	松弛迭代法
计算用时	2.1186s	64.0966s	0.087826s	0.086456s
相对误差	-	0.040806	0.059755	0.036438

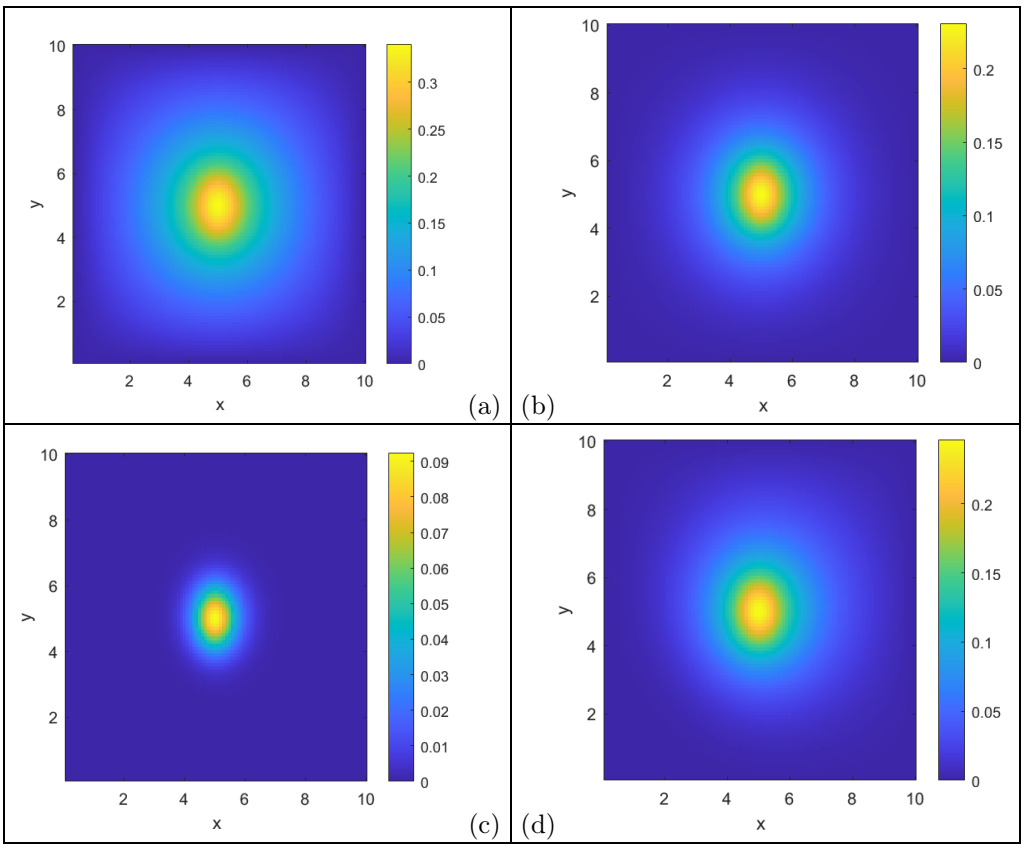


图 4. 给定内部热源功率空间分布的稳定温度场模型-四方法的计算值， (a)普通差分法， (b)差分+SmartJ，(c)GaussSeidel迭代，(d)松弛迭代法.

测试代码详见附件*Test2.m*.

4.1.3 结论

从以上两个实例中可以看出， 相较于普通差分法， 使用了Smart-Jacobi法求解矩阵的差分法计算用时反而增加， 且稳定性较差。 而另外两种迭代法都是比普通差分法更好的选择， 其中又数松弛迭代法最好，在计算速度和稳定性上都达到了最优。

4.2 第二类边界条件的松弛法的验证：电介质周围电势

在匀强电场$\vec{E}=40\vec{x}$ 中放置一近方形电介质，电介质电荷密度分布为

$$\frac{q}{\varepsilon_0}(x,y)=-25\tanh(10(x-4))\tanh(10(y-4))$$

如图5.a所示。稳定电势满足如下方程：

$$\frac{\partial^2 T(x,y)}{\partial x^2}+\frac{\partial^2 T(x,y)}{\partial y^2}+q(x,y)=0 \tag{9}$$

源项函数：$q(x,y)=25\tanh(10(x-4))\tanh(10(y-4))$.

边界条件：$\frac{\partial T}{\partial y}(x,0)=\frac{\partial T}{\partial y}(x,10)=0,\frac{\partial T}{\partial x}(0,y)=\frac{\partial T}{\partial x}(y,10)=-40.$

方法参数：$h=0.1$, 迭代的容许误差$e=10^{-3}$, 松弛变量$\omega=1.8$.

数值计算的结果如图5(b)和5(c)所示。

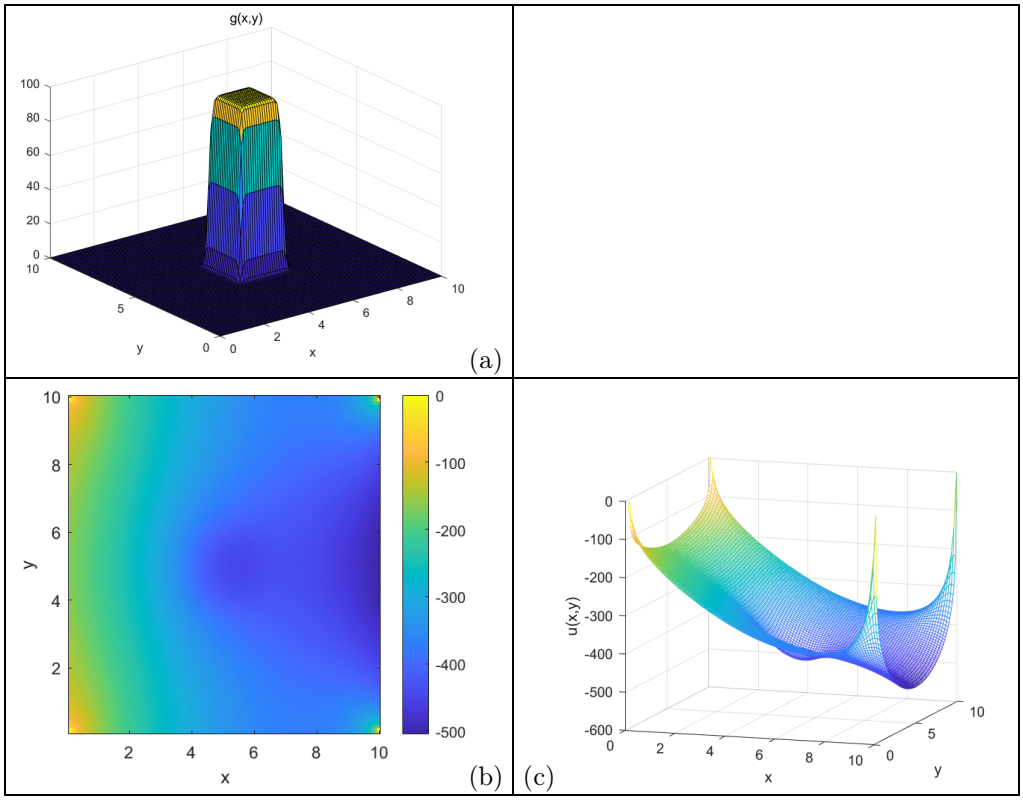


图 5. 电介质周围电势的数值计算

可见在四个角落处的迭代不收敛，这也是我们算法的一个缺陷，需要进一步完善。

测试代码详见附件$S3.m$.

4.3 （拓展）二维 Poiseuille flow 的求解

Navier-Stokes (NS)方程是表述流体运动的基本偏微分方程之一，其表达式为：

$$\rho\left(\frac{\partial \boldsymbol{v}}{\partial t}+(\boldsymbol{v}\cdot\nabla)\boldsymbol{v}\right)=-\nabla p+\mu\nabla^2\boldsymbol{v}+\boldsymbol{f} \tag{10}$$

其中，ρ为流体的密度，$\boldsymbol{v}$是流体的速度场向量，t是时间，p是压力场，μ是流体的动力粘度，$\boldsymbol{f}$是外部施加在流体上的体积力。该方程考虑了包含质量、能量、动量守恒等诸多因素。我们在这里不过多讨论类似湍流、压缩性等复杂系统，而是选择Poiseuille flow进行研究，即只考虑流体在流动过程中处于稳态，不可压缩，忽略NS方程中非线性压力梯度项，仅保留粘度和惯性的贡献。约化后的方程形式为

$$\nabla \cdot \boldsymbol{v} = 0 \qquad \text{(连续型方程)} \tag{11}$$

$$-\nabla \vec{p} + \mu \nabla^2 \boldsymbol{v} = 0 \text{ (动量守恒方程)} \tag{12}$$

方程 11 为连续型方程，确保了流体在任何给定区域内的流入和流出的量相等。这个方程确保了在流动过程中流体不会在任何地方堆积或稀疏。

方程 12 为动量守恒方程，表示流体的运动受到压力梯度和流体自身粘性阻力的影响。当压力梯度为x, y的函数$p(x,y)$时， 此时二维Poiseuille flow 的控制方程正是泊松方程（即，$g(x,y) = \frac{\nabla p(x,y)}{\mu}$），我们可以使用前面得到的松弛法迭代对$p(x,y)$取不同形式的条件下进行检验。为了简化过程，我们取流体动力粘度$\mu = 1$,即$\nabla p = g$。在二维 Poiseuille 流动中，压力沿流动方向呈线性变化，而在横向方向上基本不变：

$$p(x) = p_0 - \frac{\Delta p}{L} x \tag{13}$$

其中，Δp是液体流过通道的总压降，L是流动长度。 $g = \nabla p = \text{Constant}$。通过设置一系列的常数$g$，我们可以看到流体速度场强随着压强梯度的变化而变化。

同时设置

边界条件：$T(0,y) = 100$，$T(10,y) = -100$，$T(0,y) = T(y,10) = 0$.

(水流从x=0流入，从$x = 10$流出并限制端口的流量为100, 在y方向上封口，即不会有水流从两侧流出)

方法参数：$h = 0.1$, 迭代的容许误差$e = 10^{-3}$, 松弛变量$\omega = 1.8$.

我们使用$Applied_in_Poiseuille_flow.mlx$。测试$g = 100, 200, 300, 400, 500$的稳态流速场情况，结果如下：

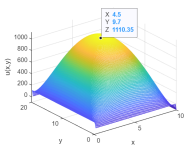
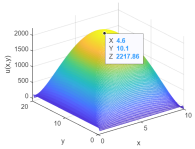
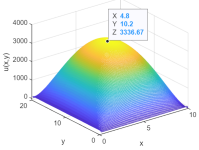
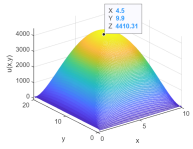
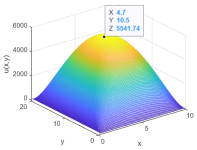
g	100	200	300	400	500
					
$v_{\max}$	~1110	~2217	~3336	~4410	~5540

图 6. 流速随着压强梯度的增长而线性增长

我们发现随着压力梯度的增大，通道内的流速接近线型增加，这与实际情况相符。表明了我们先前的近似并不会对结果产生过多影响。并且通过松弛法迭代能让我们在较短时间内得到相关的演算结果，为进一步精细化模拟提供了大致的预模拟。

测试代码详见附件$Applied_in_Poiseuille_flow.mlx$.

5 总结与展望

本项目通过对泊松函数的几种数值求解方法进行实现和分析，最终出在解决泊松方程初值问题中，松弛法迭代在用时和误差收敛速度上都好过其他三种方法。并使用松弛法迭代来模拟二维 Poiseuille flow，得到了与实际相符合的结论——压力梯度越大，通道内流速越大的结论，这也验证了松弛法解泊松方程的优越性。

不过，我们的工作还有许多不足之处。比如对于泊松方程虽能够使用松弛迭代的方式取得较好的效果，但我们尚未考虑与时间作用因素，以及目前对于一般的流体运动模拟，不仅要涉及到普通的准直或环状边界条件，可能还要涉及到复杂形状，而且由于边界与流体的接触情况多种多样，大部分流体的控制方程已经不能由泊松方程来约化。我们在完成项目的过程中，也了解到了很多复杂流体模拟的相关优秀方法，例如Lattice Boltzmann Method, (LBM)[2]。 如果未来有时间改进，我们或许可以对已有的松弛法在二维不含时的基础上， 加入NS方程中的含时项， 以及尝试实现D_2Q_9[3]模型（**基于松弛迭代法**）并模拟流体中的湍流现象等等。

参考文献

[1] 彭芳麟. *计算物理基础*. 高等教育出版社, 2010.

[2] Yuanxun Bao 与 Justin Meskas. Lattice boltzmann method for fluid simulations. 2011.

[3] Shiyi Chen, Daniel Martnez, 与 Renwei Mei. On boundary conditions in lattice Boltzmann methods. *Physics of Fluids*, 8(9):2527–2536, sep 1996.